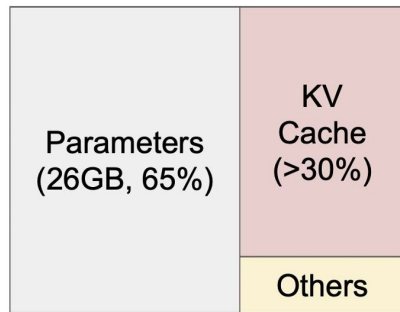
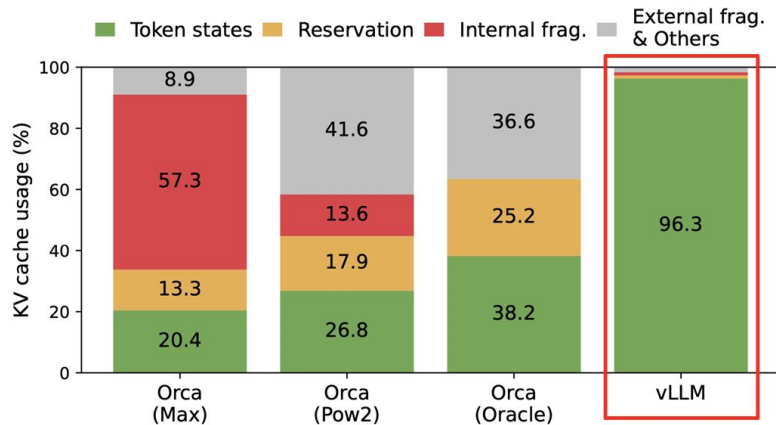

vAttention Overview

Background

- Paged Attention Slides
- KV Cache Issue
 - Not Fixed size



NVIDIA A100 40GB



*Source : PagedAttention

Motivation for vAttention

- Not using PagedAttention -> Low GPU memory utilisation
- Using PagedAttention
 - Kernel Re-writing
 - Redundancy by Translation
 - Runtime Overhead

Redundancy

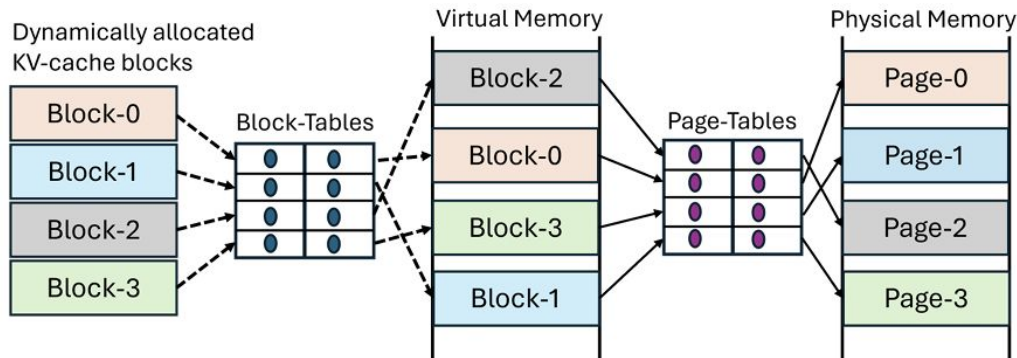
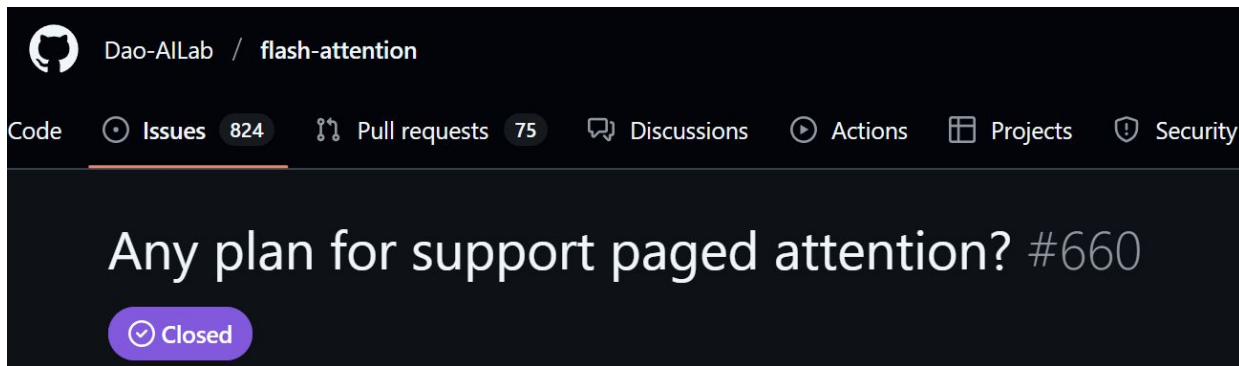


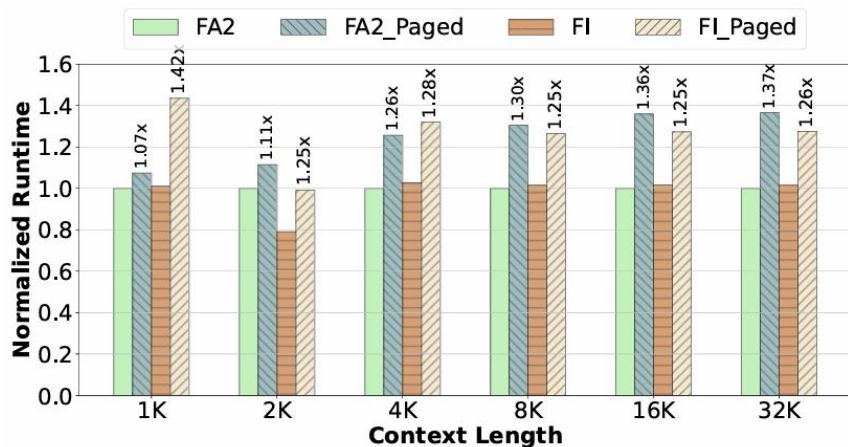
Figure 1. PagedAttention involves two layers of memory management: one in user space and one in OS kernel space.

Rewriting Kernels

- Attention assumes Virtual memory contiguous
- Needs custom support



Performance



System/library and issues related to PagedAttention

vLLM [51]: Pioneered PagedAttention. Despite being in an actively maintained code repository, vLLM's PagedAttention kernel is up to 2.8× slower than FlashAttention-2 (Table 7). Furthermore, changing block size changes the execution time of the kernel by as much as 1.9× (Figure 3).

FlashAttention-2 [42]: PagedAttention-based prefill kernel is up to 37% slower than the non-paged kernel (Figure 2) while the decode kernel is up to 12% slower. Initial attempts to add paging support failed unit tests [7].

FlashAttention-3 [67] / SDPA in cuDNN-9 [59]: State-of-the-art attention kernels for NVIDIA Hopper architecture did not support PagedAttention when released.

TensorRT-LLM [6]: Serving throughput dropped by more than 10% in a Python front-end [5]. Recommends using the C++ front-end. Even with C++, we observe up to 5% higher latency in some cases with PagedAttention.

FlashInfer [77]: PagedAttention-based prefill kernel is up to 42% slower than the non-paged kernel (Figure 2).

vAttention Working

Allocate large contiguous Virtual memory buffers

&

Defer Physical Memory Allocation

Pre reserve Virtual Memory

- We have 128 TB available !
- 2 X N buffers
- Buffer Size = $B * L * H * D * P$

Eg. Yi-34B , $N = 60, H = 2, D = 128, P = 2, B = 500$

Buffer Size = 100 GB,

Total KV Cache Size = $2 * 60 * 100 \text{GB} = 12 \text{TB}$, 10% used

Symbol	Definition
N	Number of layers on a particular worker
H	Number of KV heads on a particular worker
D	Dimension of each attention head
P	Number of bytes needed to store one element
B	Maximum batch size
L	Maximum context length supported by the model
L'	Context length of a request seen thus far

Table 2. Notations used in the paper.

Dynamic Memory Management

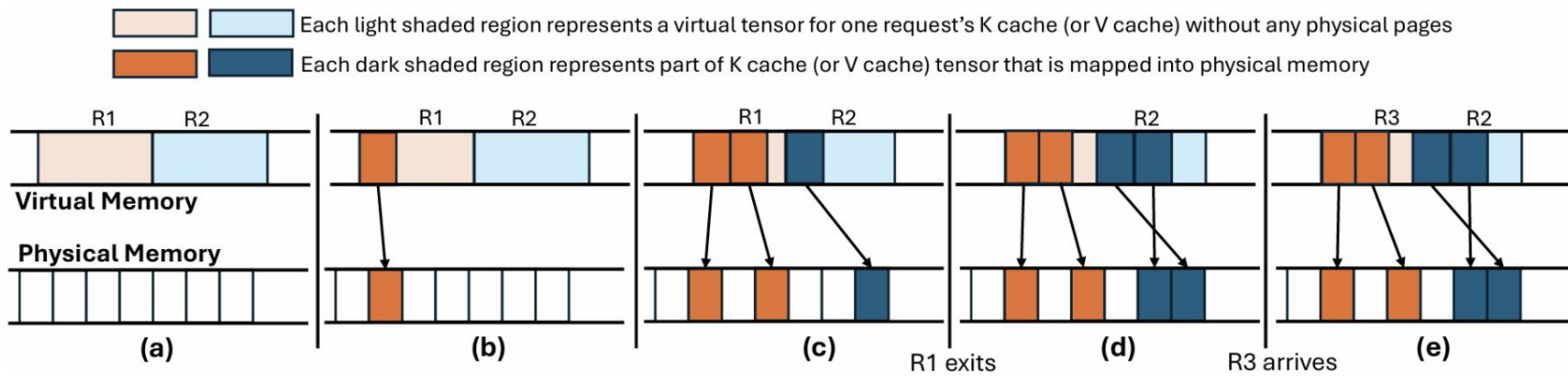


Figure 5. Dynamic memory management in vAttention for a single K cache (or V cache) tensor. (a) shows a virtual tensor for a batch of two requests with no physical memory allocation yet. (b) R1 is allocated one physical page. (c) R1 is allocated two pages and R2 is allocated one page. (d) R1 has completed but vAttention does not reclaim its memory (deferred reclamation). (e) when R3 arrives, vAttention assigns R1's tensor to it which is already backed by physical memory.

Cuda API's Used

CUDA VM APIs	vAttention VM APIs	Description	Latency (microseconds)			
			64KB	128KB	256KB	2MB
cuMemAddressReserve *	vMemReserve *	Allocate a buffer in virtual memory	18	17	16	2
cuMemCreate *	vMemCreate *	Allocate a handle in physical memory	1.7	2	2.1	29
cuMemMap	vMemMap	Map a physical handle to a virtual buffer	8	8.5	9	2
cuMemSetAccess	-	Enable access to a virtual buffer	-	-	-	38
cuMemUnmap	-	Unmap physical handle from a virtual buffer	-	-	-	34
cuMemRelease *	vMemRelease *	Free physical pages of a handle	2	3	4	23
cuMemAddressFree *	vMemFree *	Free a virtual memory buffer	35	35	35	1

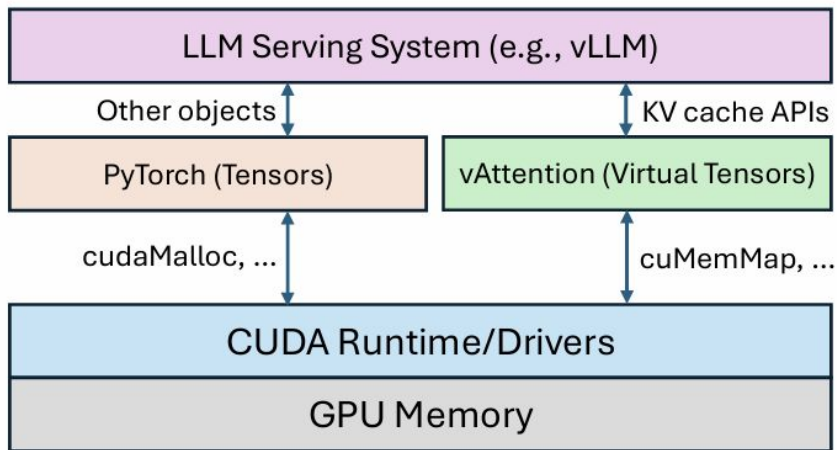


Figure 6. An LLM serving system interacts with vAttention for KV cache management with a set of simple APIs listed in [Table 4](#). All other memory objects (e.g., activations) are allocated by the standard PyTorch caching allocator.

Algorithm 1 Using vAttention in a serving framework.

```
1: max_batch_size  $\leftarrow$  B
2: cache_seq_len  $\leftarrow$  [0]*B
3: req_batch_idx  $\leftarrow$  dict()
4: vattention.init(config_params)
5: while !request_pool.is_empty() do
6:   for  $R_i$  in new_requests do
7:     if can_schedule( $R_i$ ) then
8:       idx  $\leftarrow$  vattention.alloc_reqid()
9:       req_batch_idx[ $R_i$ ]  $\leftarrow$  idx
10:      cache_seq_len[idx]  $\leftarrow$  prompt_len( $R_i$ )
11:    end if
12:  end for
13:  vattention.step(cache_seq_len)
14:  model.forward()
15:  for  $R_i$  in active_requests do
16:    idx  $\leftarrow$  req_batch_idx[ $R_i$ ]
17:    if is_complete( $R_i$ ) then
18:      cache_seq_len[idx]  $\leftarrow$  0
19:      vattention.free_reqid(idx)
20:    else
21:      cache_seq_len[idx] += 1
22:    end if
23:  end for
24: end while
```

APIs	Description
init	Initializes vAttention with model parameters. arguments: N, B, L, H, D, P , page-group-size. return value: a list of KV cache tensors.
alloc_reqid	Allocates an unused reqId and marks it active arguments: None return value: an integer reqId
free_reqid	Frees a reqId and marks it inactive arguments: an integer reqId return value: None
step	Ensures KV cache tensors are backed by physical pages up to the current context length of each active request arguments: an array of size B containing sequence lengths of each reqId return value: 0 (success), -1 (failure).

Optimisations

- Latency Hiding
 - Overlapping memory allocation with compute
 - Deferred Reclamation
- Mitigating Internal Fragmentation

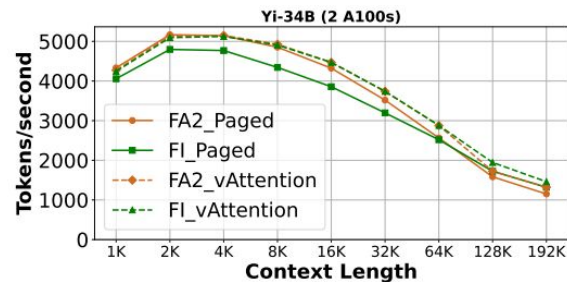
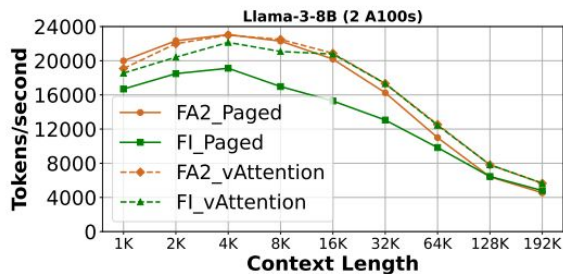
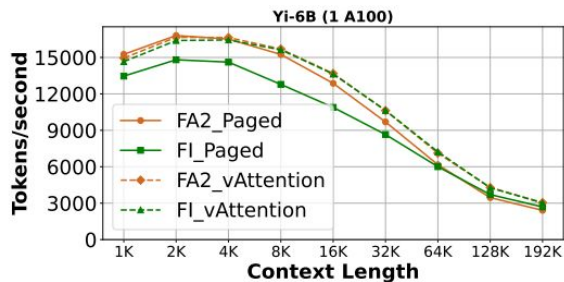
Evaluation

Methodology

- FlashAttention-2 v2.5.9
- FlashInfer v0.4.0
- Both paged and non-paged

Model	Hardware	# Q Heads	# KV Heads	# Layers
Yi-6B	1 A100	32	4	32
Llama-3-8B	2 A100s	32	8	32
Yi-34B	2 A100s	56	8	60

Prefill Evaluation



*vLLM not used since it doesn't have a prefill kernel

Prefill Evaluation

- VAttention Faster
- Speedup in Attention

Model	Context Length	FlashAttention-2		FlashInfer	
		Paged	vAttention	Paged	vAttention
Yi-6B	64K	10.6 (7.0)	9.1 (5.5)	10.9 (6.0)	9.1 (5.4)
	128K	37.9 (30.3)	30.5 (23.1)	35.4 (25.4)	30.7 (23.3)
	192K	81.5 (70.0)	64.6 (53.6)	73.0 (58.3)	65.1 (53.6)
Llama-3-8B	64K	6.0 (3.4)	5.2 (2.7)	6.7 (3.0)	5.3 (2.8)
	128K	20.4 (15.4)	16.8 (11.6)	20.3 (12.8)	16.8 (11.4)
	192K	43.3 (35.6)	34.8 (26.9)	40.9 (29.7)	34.7 (26.7)
Yi-34B	64K	25.5 (13.2)	22.8 (10.3)	26.0 (11.2)	22.7 (10.1)
	128K	82.8 (56.9)	68.4 (43.2)	76.0 (46.7)	67.4 (42.5)
	192K	170.7 (131.8)	136.9 (98.8)	148.8 (104.7)	134.6 (96.5)

Table 6. Prefill completion and attention (in parenthesis) time with different attention back-ends (unit: seconds).

Decode Evaluation

- Long context 16K used to see effect of Attention time
- Configs
 - vLLM ~ Baseline
 - FA2-Paged
 - FA2-Vattention
 - FI-Paged

*FlashInfer's non-paged decode kernel has significantly higher latency (up to 14.6×) compared to all these other kernels

Decode Throughput

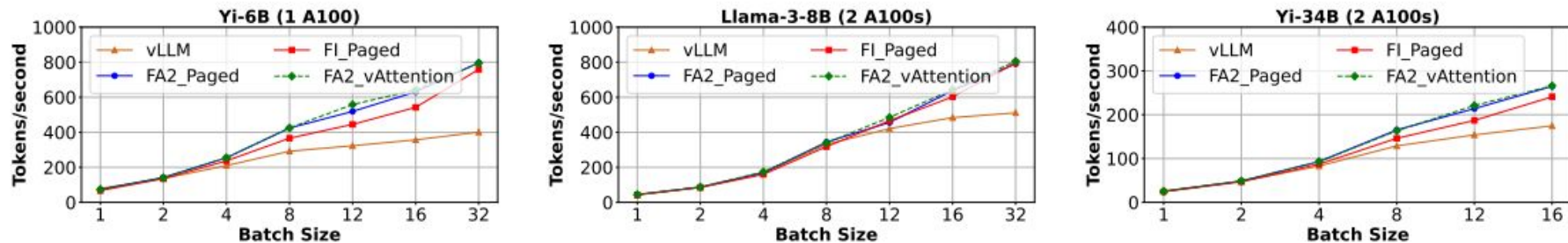


Figure 8. Decode throughput. FA2_vAttention is on par with FA2_Paged (note the overlapping lines) which is the best among all PagedAttention based alternatives, while outperforming FI_Paged and vLLM.

Model	BS	vLLM	FA2_Paged	FI_Paged	FA2_vAttention
Yi-6B	16	32.3	11.5	15.2	11.3
	32	64.1	25.5	25.4	25.3
Llama-3-8B	16	17.8	11.9	12.1	11.8
	32	35.3	25.4	23.23	25.3
Yi-34B	12	41.4	17.4	24.1	17.4
	16	55.1	21.7	28.8	21.8

Table 7. Total latency of attention kernel (sum of all layers) per decode iteration (in milliseconds, BS = batch size).

E2E Performance

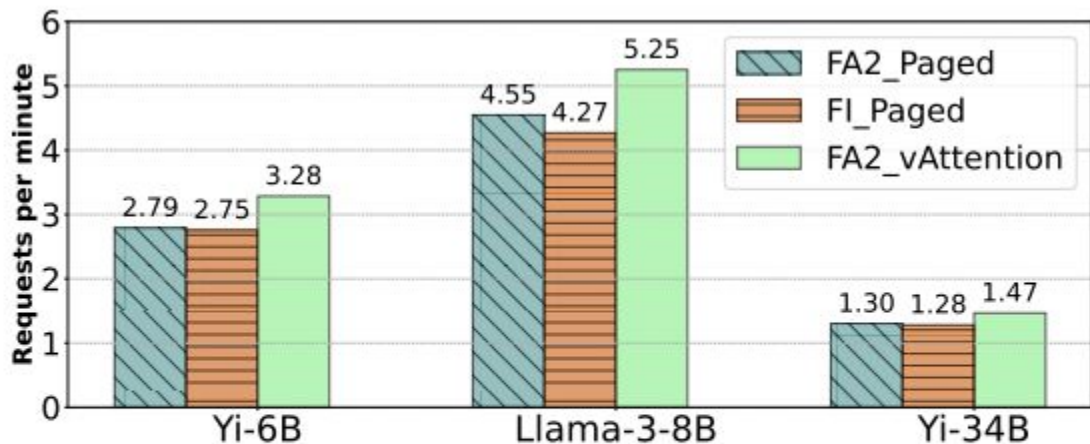


Figure 9. Offline inference throughput while serving long-context requests from the arXiv-Summarization dataset [11].

E2E Performance

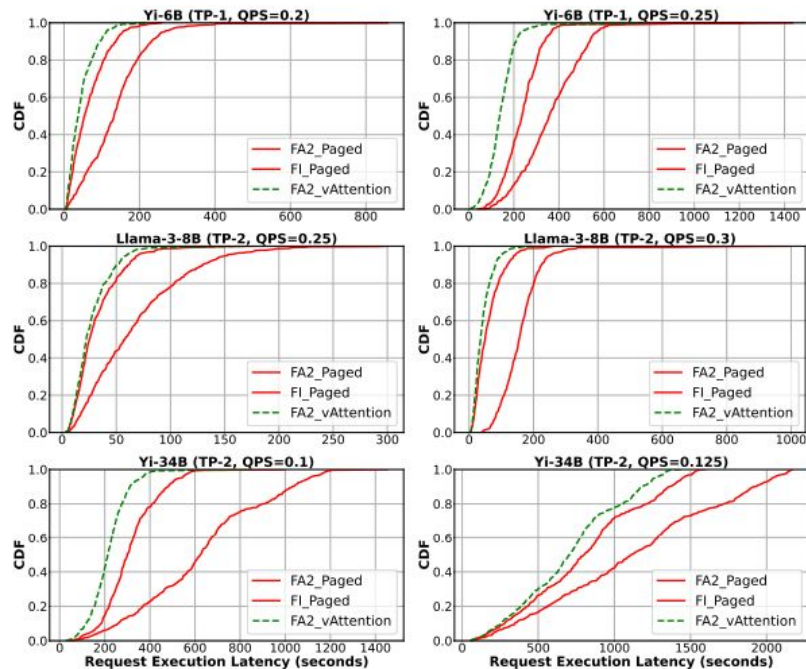


Figure 10. CDF of end-to-end request execution latency in online inference under varying load.

Ablation Studies

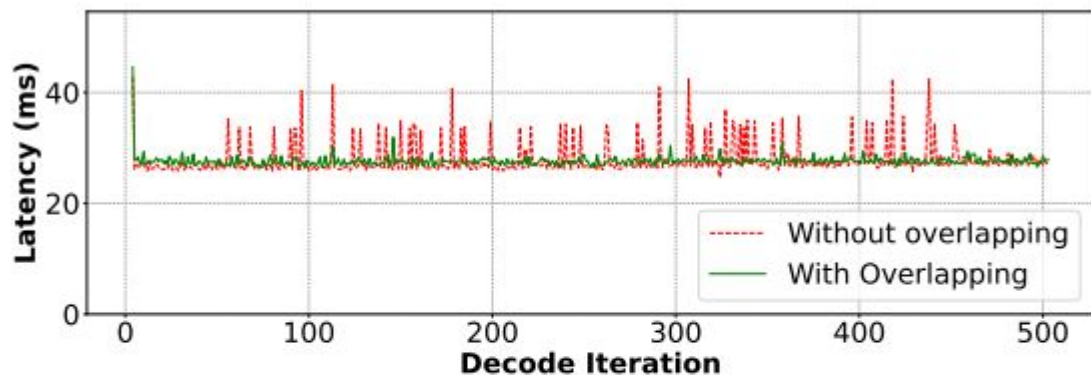


Figure 12. Latency of decode iterations with and without overlapping memory allocation with compute (batch size=32, context length=4K–8K, model: Llama-3-8B)

Deferred Reclamation

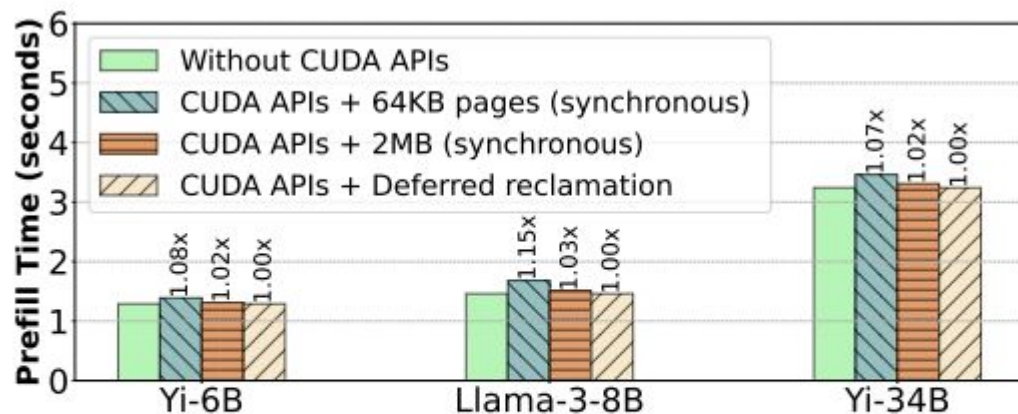


Figure 13. Prefill completion time of a single prompt of 16K tokens with different memory allocation strategies.

Page Size on Runtime

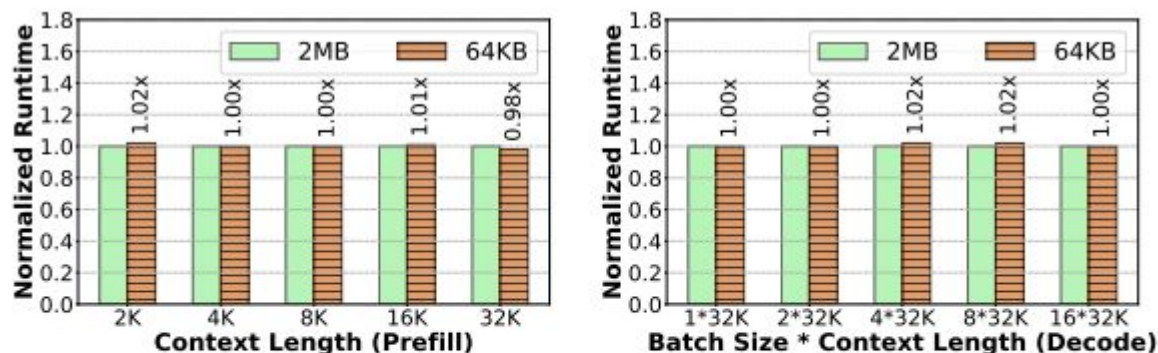


Figure 14. Effect of page size on the runtime of FlashAttention-2's prefill (left) and decode (right) attention kernels (model: Llama-3-8B).